Elyza C. Macapanas

Sameryishin N. Cadelina

Section-C

NOVEMBER 11, 2026

Java OOP Laboratory Exercise: Campus Café Ordering System

If Statement

Used in "Pastry.computePrice()" and "Meal.computePrice()" to add extra charges when a pastry is warmed or a meal is upsized.

If-else Statement

Used to check customer choices and display the correct description. For example, it displays whether a pastry is warmed or served normally.

Classes and Objects

Created seven classes: "Meal", "Beverage", "Pastry", "MenuItem", "Customer", "Order", and "CafeApp". These classes serve as blueprints for the objects used in the program.

Attributes and Behaviors

Used variables to store information such as customer names, item prices, and order details. Methods perform actions such as calculating prices and printing receipts.

Object Interaction

Connected the "Order", "Customer", and "MenuItem" objects. The Order class manages the customer's selected items, calculates the total, and updates loyalty points.

Java Naming Conventions

Applied PascalCase for class names, camelCase for methods and variables, and UPPER_SNAKE_CASE for constants to make the code organized and readable.

Default Constructor

Provided default constructors in the MenuItem subclasses and Customer class to allow objects to be created without initial values.

Parameterized Constructor

Used parameterized constructors to initialize objects with specific information, such as item names, prices, customer details, and order information.

Constructor Overloading

Implemented three constructors in the Customer class to allow customers to be created using different sets of information.

Public, Private, and Protected

Used public methods for accessible operations, private variables to protect data, and protected fields in MenuItem to allow its subclasses to access shared information.

Getters, Setters, and Data Hiding

Used private variables with getters and validated setters to protect information and prevent invalid values, such as negative item prices.

Method Overloading

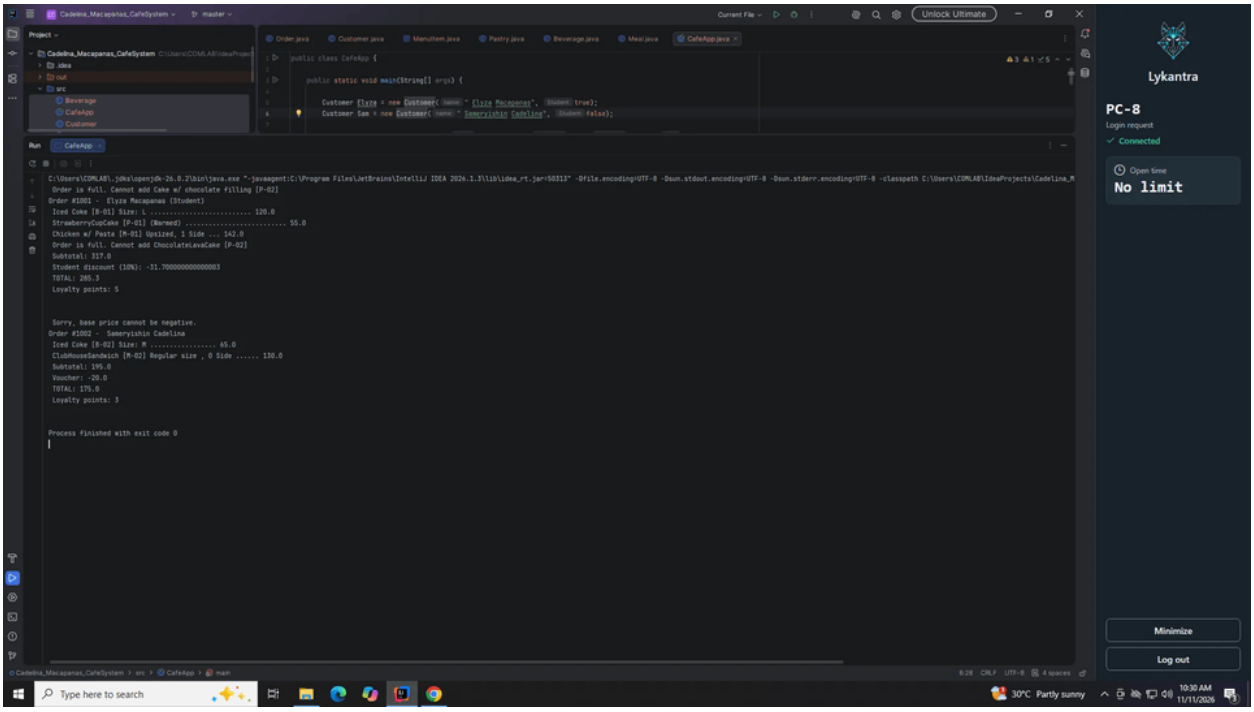
Created methods with the same name but different parameters, such as "Customer.addPoints(int)" and "Customer.addPoints(double)", and "Order.computeTotal()" and "Order.computeTotal(double)".

Method Overriding

Overrode "computePrice()" and "getDescription()" in Beverage, Pastry, and Meal to provide different calculations and descriptions for each menu item.

Abstraction Used

MenuItem as an abstract class with an abstract "computePrice()" method. This requires its subclasses to provide their own price calculations.



Encountered and Solutions One of the challenges encountered while developing the system was displaying the message "Order is full. Cannot add [item]." in the correct position. Initially, the message appeared in the wrong part of the output, affecting the arrangement of the receipt. To solve this problem, the order limit checking and message display were adjusted so that the system could properly identify when the maximum number of items had been reached and display the error message in the intended position. Another challenge was ensuring that invalid data, such as negative item prices, would not be accepted. This was addressed by using validation in setter methods to prevent incorrect values from being assigned.